

DA 460: An Overview of Models for Kirby's Dream Land

Lucas Hasting

December 2025

Contents

1	Overview	3
2	Summary Statistics	5
3	The Models	6
3.1	Decision Tree Preview	7
4	Model Results	9
5	Python Programs Used	11
6	Sources	23

1 Overview

To begin, Kirby's Dream Land is a video game on the Nintendo Gameboy with the genre of arcade platformer. The goal of this project is to predict the move Kirby will make on any given frame of the game. A frame consists of many variables including screen data, tile data (for the current level), game state, visual scroll (x/y position), Kirby's (x/y) position on the screen, and the boss's current health. The models used for determining this question could help determine what variables dictate the move the player would make, and could provide insight on how to better play the game. These variables mentioned in the frame data would be used to make said prediction.

First, the screen data, visual scroll, and Kirby's position on the screen could inform the player how many enemies are on the screen and their location to better predict where the player should avoid. Next, tile data could be used to better determine what tiles Kirby should avoid, for example, different stages make use of different tiles and one stage in particular has a lot of projectiles coming from trees, so a prediction could be made to stay above the trees on that specific stage. Also, game state, and the boss's current health could be used to make different predictions on the move based on what it is, for example, when Kirby gets power-ups the game state changes to a specific encoding, and when the boss health is greater than 0, the screen locks its position and Kirby has to fight the boss.

The data for each frame was gathered in python using a Gameboy emulator known as gambatte [3]. The API "gym retro" was used in order for python to interact with the emulator and retrieve data from the memory addresses. The memory addresses that were gathered are documented online [1] and were verified using gym retro. A python script was used to create a json file that includes the addresses for the variables mentioned. The variables themselves are mixed nominal and numeric, and the output that we are trying to predict is nominal, more specifically, we are trying to predict what button(s) on the gameboy would be pressed at any given time until the game is finished. Each button/button combination that would make Kirby move is included as a class, and the option to make no move is also included as a class. The data comes from a play-through of the game by the



Figure 1: Kirby Scroll 1

author of this paper and the data can be found online [2], on the author's GitHub repository for this project. Every frame has complete observations, meaning there are no missing values, with a total of 85,067 observations.

The input variables will be described. Next, Screen Data includes 160 numeric variables describing what is on the screen. Moreover, Tile Data includes 164 numeric variables describing the tiles in the current stage. Also, Game State is a nominal variable describing the moves the player can make, e.g., a power up allows the player to make different/similar moves with the same button(s), and an example of this is shown in figures 1 and 2 (the alternating color indicates a powerup). Then, the visual scroll (`kirby_x_scroll`, and `kirby_y_scroll`) includes 2 numeric variables describing the rate at which the screen is scrolling across either the x-axis or y-axis, and an example of this is shown in figures 1 and 2. Moreover, Kirby's x/y position (`kirby_x` and `kirby_y`) are numeric variables describing where Kirby is on the screen. Next, the boss's health keeps up with the amount of health the current boss has (a boss health of 0 means that there is no boss present). Finally, other variables that could be useful for prediction is enemy data, placement of tiles, and placement of projectiles. Although, some of this may be included as part of screen data, screen data is encoded as binary data and is hard to decipher entirely what we get out of it, the data for these variables might be saved somewhere in memory, and is hard to find. In the next section we will look at summary statistics for the data.



Figure 2: Kirby Scroll 2

2 Summary Statistics

First, the data file can be found online [2]. The summary statistics for each variable are shown below:

Variable	Mean	Standard Deviation	Q_1	Q_2	Q_3	Min	Max
Screen Data	52.7	61.8	0	32	97	0	255
Tile Data	0.99	7.27	0	0	2	0	205
kirby_x	11650.17	18553.15	98	210	18307	0	65535
kirby_y	5591.3	14257.68	84	151	231	0	65533
kirby_x_scroll	73.53	35.8	47	76	89	8	152
kirby_y_scroll	5.27	6.18	0	3.2	9.7	0	25.5
boss_health	2.13	2.93	0	0	4	0	10

Table 1: Numerical Variables - Summary Statistics

Class	Count
1	74360
2	269
3	63
4	10
5	218
6	8003
7	16
8	1929
11	199

Table 2: Game State - Count

Class	Count
NONE	33643
RIGHT	19604
LEFT	6867
UP	2669
DOWN	1368
A	1042
B	5087
B-RIGHT	3171
B-LEFT	1029
A-RIGHT	2275
A-LEFT	642
UP-RIGHT	3017
UP-LEFT	1562
DOWN-RIGHT	13
DOWN-LEFT	78

Table 3: Move Made (Outcome Variable) - Count

3 The Models

The models that are used for prediction are a Decision Tree and k -Nearest Neighbor (k -NN) Classification. The game state variable was transformed into binary dummy variables, and was then further transformed using a min-max normalization with a min of 0 and max of 255 (which is the range of a byte). Next, the kirby_x_scroll and kirby_y_scroll variables were transformed using min-max normalization in the same range (0 through 255), since it was represented with two bytes.

These transformations were done since every other variable was in the range of one byte. The decision tree is too large to show, but can be viewed partially at [5]. A preview of [5] is shown in section 3.1. Next, the value of k that was chosen for k -NN was 7, and the max depth of the decision tree chosen was 17. Also, since the all the data is in the same range, euclidean distance was used as the distance metric. The value of the tree's depth and the value of k were chosen using the holdout method [4] along with a parameter search. The holdout method was used to ensure the parameter chosen does not result in a model that is over-fitted. The threshold for the generalization gap that was chosen was 5% (it was chosen simply because it was a small error percentage). Moreover, the training and test sets were split (randomly) using (approximately) a 2/3 and 1/3 split respectively. Moreover, the training set has 59,546 observations, and the test set has 25,521 observations.

3.1 Decision Tree Preview

A preview of [5], which is an ASCII representation of the decision tree is shown below:

```

1 |--- screen1_3 <= 24.50
2 | |--- screen1_2 <= 3.00
3 | | |--- screen1_2 <= 1.00
4 | | | |--- state_6 <= 127.50
5 | | | | |--- kirby_x_scrol <= 42.50
6 | | | | | |--- screen3_1 <= 120.00
7 | | | | | | |--- screen7_2 <= 132.00
8 | | | | | | | |--- screen16_3 <= 16.50
9 | | | | | | | | |--- class: RIGHT
10 | | | | | | | | |--- screen16_3 > 16.50
11 | | | | | | | | |--- kirby_y <= 0.61
12 | | | | | | | | | |--- screen4_1 <= 100.00
13 | | | | | | | | | | |--- class: NONE
14 | | | | | | | | | | |--- screen4_1 > 100.00
15 | | | | | | | | | | |--- class: DOWN

```

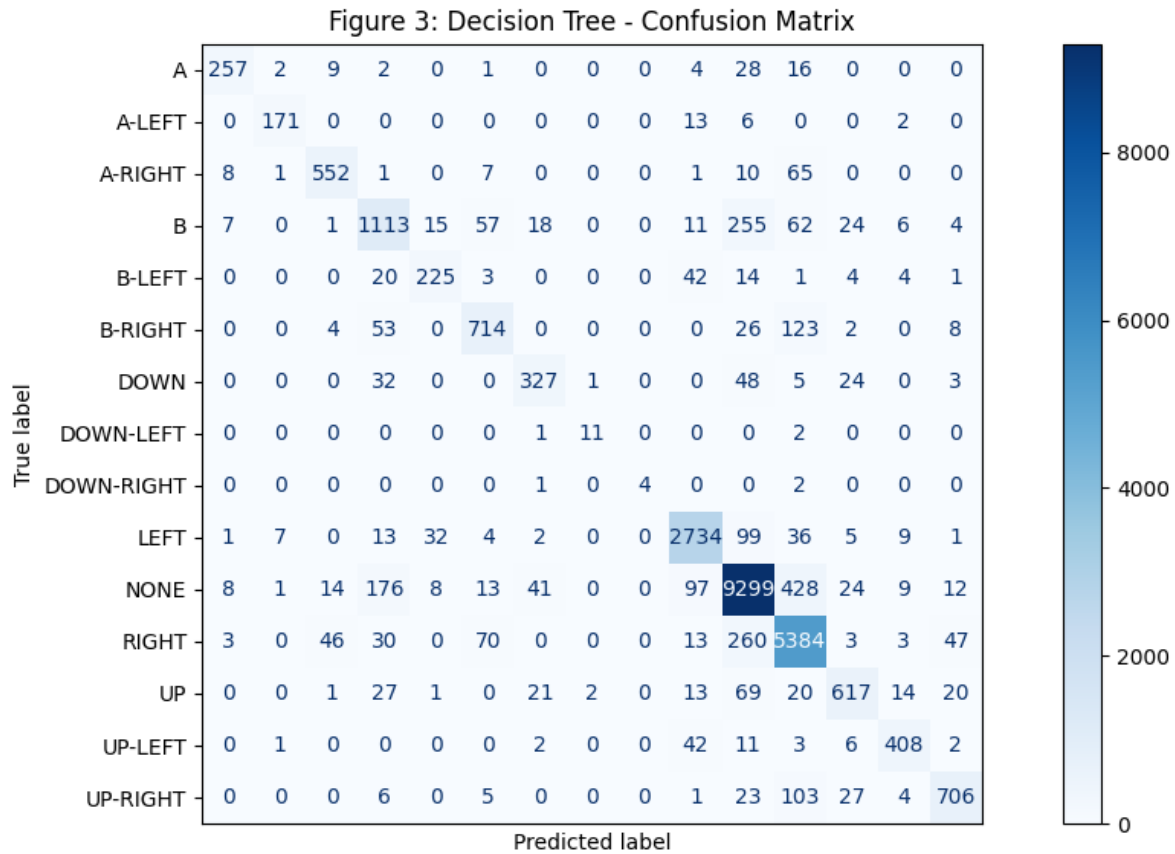
```

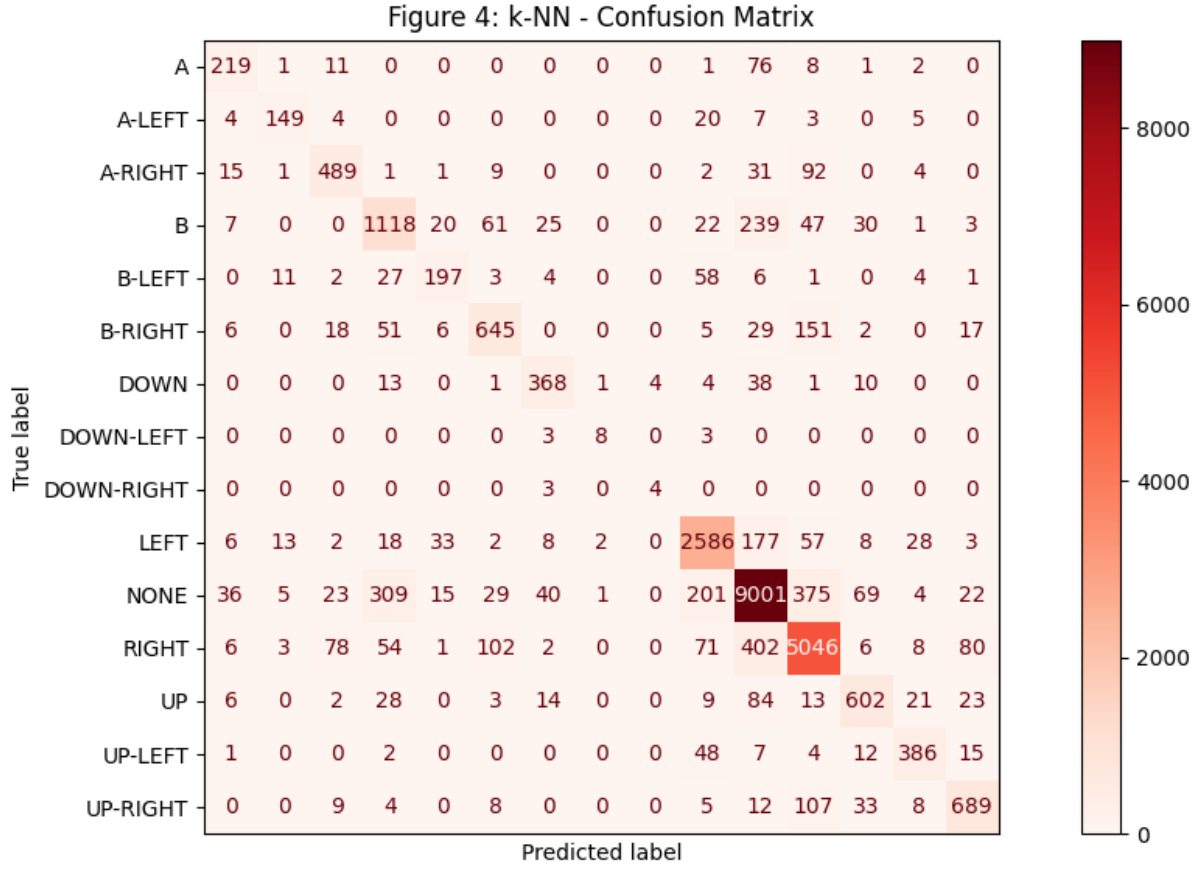
16 | | | | | | | | |--- kirby_y > 0.61
17 | | | | | | | | |--- screen4_0 <= 40.50
18 | | | | | | | | |--- kirby_y_scrol <= 54.50
19 | | | | | | | | |--- truncated branch of depth 3
20 | | | | | | | | |--- kirby_y_scrol > 54.50
21 | | | | | | | | |--- truncated branch of depth 2
22 | | | | | | | | |--- screen4_0 > 40.50
23 | | | | | | | | |--- screen8_3 <= 17.00
24 | | | | | | | | |--- class: NONE
25 | | | | | | | | |--- screen8_3 > 17.00
26 | | | | | | | | |--- truncated branch of depth 2
27 | | | | | | |--- screen7_2 > 132.00
28 | | | | | | |--- screen14_1 <= 63.50
29 | | | | | | |--- screen9_2 <= 150.00
30 | | | | | | | | |--- kirby_y_scrol <= 81.00
31 | | | | | | | | |--- screen21_1 <= 54.50
32 | | | | | | | | |--- class: NONE
33 | | | | | | | | |--- screen21_1 > 54.50
34 | | | | | | | | |--- truncated branch of depth 2
35 | | | | | | | | |--- kirby_y_scrol > 81.00
36 | | | | | | | | |--- class: A
37 | | | | | | | | |--- screen9_2 > 150.00
38 | | | | | | | | |--- class: A
39 | | | | | | | |--- screen14_1 > 63.50
40 | | | | | | | |--- kirby_y_scrol <= 41.00
41 | | | | | | | | |--- class: NONE
42 ...

```


4 Model Results

First, the confusion matrices for each model are shown below:





The accuracy for the decision tree is computed below to be 88.25%:

$$\begin{aligned}
 & \frac{257 + 171 + 552 + 1113 + 225 + 714 + 327 + 11 + 4 + 2734 + 9299 + 5384 + 617 + 408 + 706}{25521} \\
 &= \frac{22522}{25521} \\
 &\approx 88.25\%.
 \end{aligned}$$

Moreover, the accuracy for the k -NN model is computed below to be 84.27%:

$$\begin{aligned}
 & \frac{219 + 149 + 489 + 1118 + 197 + 645 + 368 + 8 + 4 + 2586 + 9001 + 5046 + 602 + 386 + 689}{25521} \\
 &= \frac{21507}{25521} \\
 &\approx 84.27\%.
 \end{aligned}$$

Next, the model's results were surprising. We can see from the decision tree, that the screen data

contributed heavily to the prediction (it was the main predictor), and since we do not know what is included with screen, it was an informative result of the model. Moreover, the k -NN model's accuracy was surprising, since there are 338 total variables (including the dummy variables) we expect the model's accuracy to be bad by the curse of dimensionality, but instead we get a model that is more than 70% accurate.

Next, further studies could explore additional models. Moreover, variables can be added with further reverse engineering (the process of extracting information out of software without the source code), which may contribute to either model.

5 Python Programs Used

The python programs used in this project with a corresponding description are all listed in the following section, these files can be downloaded at [2]. First, we have the **get_data.py** file which creates the **data.json** file which contains the memory addresses the next few programs will make use of:

```
1 #Name:          Lucas Hasting
2 #Class:         DA 460
3 #Date:          12/7/2025
4 #Instructor:    Dr. Imbrogno
5 #Description:    Setup json file that contains memory address of
                   interest
6 #Sources:        https://datacrystal.tcrf.net/wiki/Kirby%27s_Dream_Land/
                   RAM_map
7
8 screen = 49152 # screen begining address
9
10 i = 1
11 cont = screen
12
13 file = ""
14 file += '\n\t"info": {\n'
15
16 #screen variables
17 while(i < 41):
```


Next, we have the **game_data.py** file which creates the **kdl.csv** file and that file contains the data used throughout the project:

```
1 #Name:          Lucas Hasting
2 #Class:         DA 460
3 #Date:          12/7/2025
4 #Instructor:    Dr. Imbrogno
5 #Description:    Course Project – Record game data and movements made by
                  the player of Kirby’s Dream Land
6 #              – q to quit, z = a, x = b, arrow keys to move
7 #Sources:       ChatGPT was used for syntax
8 #              https://retro.readthedocs.io/en/latest/index.html
9
10 #import libraries
11 import retro
12 import time
13 import keyboard
14 import pandas as pd
15
16 #set the frames per second
17 FPS = 60
18
19 #the starting game state
20 STATE_FILE = "begginning.state"
21
22 #create map of inputs to position in action array
23 INPUTS = {
24     "A": 8,
25     "B": 0,
26     "UP": 4,
27     "DOWN": 5,
28     "LEFT": 6,
29     "RIGHT": 7,
30     "NONE": -1,
31     "DOWN-LEFT": 1,
32     "DOWN-RIGHT": 9,
33     "UP-LEFT": 2,
34     "UP-RIGHT": 3,
35     "A-LEFT": 10,
36     "A-RIGHT": 11,
37     "B-LEFT": 12,
38     "B-RIGHT": 13
39 }
40
41 #function used to make an action in the emulator
42 def make_action(enter):
```

```

43     action = [0] * 9
44     if(INPUTS[enter] == 1):
45         action[5] = 1
46         action[6] = 1
47     elif(INPUTS[enter] == 2):
48         action[4] = 1
49         action[6] = 1
50     elif(INPUTS[enter] == 3):
51         action[4] = 1
52         action[7] = 1
53     elif(INPUTS[enter] == 9):
54         action[5] = 1
55         action[7] = 1
56     elif(INPUTS[enter] == 10):
57         action[8] = 1
58         action[6] = 1
59     elif(INPUTS[enter] == 11):
60         action[8] = 1
61         action[7] = 1
62     elif(INPUTS[enter] == 12):
63         action[0] = 1
64         action[6] = 1
65     elif(INPUTS[enter] == 13):
66         action[0] = 1
67         action[7] = 1
68     elif(INPUTS[enter] == -1):
69         pass
70     else:
71         action[INPUTS[enter]] = 1
72     return action, enter
73
74 #get all the screen/tile data (there is alot)
75 def load_other_data(info, n, typ):
76     data = []
77     for i in range(n):
78         for j in range(4):
79             data.append(info[f"{typ}{{i+1}}_{{j}}"])
80     return data
81
82
83 #load in all the data
84 def load_data(info, move):
85     screen_data = load_other_data(info, 40, "screen")
86     tile_data = load_other_data(info, 41, "tile")
87     boss_health = info["boss_health"]
88     kirby_x_scrol = info["kirby_x_scrol"]
89     kirby_x = info["kirby_x"]

```

```

90     kirby_y_scrol = info["kirby_y_scrol"]
91     kirby_y = info["kirby_y"]
92     game_state = info["game_state"]
93
94     return [*screen_data, *tile_data, game_state, boss_health, kirby_y,
95            kirby_y_scrol, kirby_x_scrol, kirby_x, move]
96
97 #write the data to a csv file
98 def write_to_csv(data, filename):
99     #create column names
100     column = []
101
102     for i in range(40):
103         for j in range(4):
104             column.append(f"screen{i+1}_{j}")
105
106     for i in range(41):
107         for j in range(4):
108             column.append(f"tile{i+1}_{j}")
109
110     column += ["game_state", "boss_health", "kirby_y", "kirby_y_scrol"]
111     column += ["kirby_x_scrol", "kirby_x", "move"]
112
113     #save file
114     df = pd.DataFrame(data, columns=column)
115     df.to_csv(filename, index=False)
116
117 #function is a new renderer that uses the FPS to determine speed
118 def new_render(env):
119     time.sleep(1/FPS)
120     env.render()
121
122 def main():
123     #start gym retro enviornment
124     env = retro.make('KirbysDreamLand-GB', STATE_FILE)
125     env.reset()
126
127     #declare a and b inputs on keyboard
128     a = "z"
129     b = "x"
130
131     #create list to store data
132     data = []
133
134     while(True):
135         #determine action from keyboard, if q is pressed, quite game
136         #and store data

```

```

135     action , move = make_action("NONE")
136     if keyboard.is_pressed('up') and keyboard.is_pressed('left'):
137         action , move = make_action("UP-LEFT")
138     elif keyboard.is_pressed('up') and keyboard.is_pressed('right')
139         :
140         action , move = make_action("UP-RIGHT")
141     elif keyboard.is_pressed('down') and keyboard.is_pressed('left')
142         ):
143         action , move = make_action("DOWN-LEFT")
144     elif keyboard.is_pressed('down') and keyboard.is_pressed('right
145         '):
146         action , move = make_action("DOWN-RIGHT")
147     elif keyboard.is_pressed('left') and keyboard.is_pressed(a):
148         action , move = make_action("A-LEFT")
149     elif keyboard.is_pressed('right') and keyboard.is_pressed(a):
150         action , move = make_action("A-RIGHT")
151     elif keyboard.is_pressed('left') and keyboard.is_pressed(b):
152         action , move = make_action("B-LEFT")
153     elif keyboard.is_pressed('right') and keyboard.is_pressed(b):
154         action , move = make_action("B-RIGHT")
155     elif keyboard.is_pressed('up'):
156         action , move = make_action("UP")
157     elif keyboard.is_pressed('down'):
158         action , move = make_action("DOWN")
159     elif keyboard.is_pressed('left'):
160         action , move = make_action("LEFT")
161     elif keyboard.is_pressed('right'):
162         action , move = make_action("RIGHT")
163     elif keyboard.is_pressed(a):
164         action , move = make_action("A")
165     elif keyboard.is_pressed(b):
166         action , move = make_action("B")
167     elif (keyboard.is_pressed('q')):
168         break
169
170     #make the action
171     ob , rew , done , info = env.step(action)
172     new_render(env)
173
174     #record the data
175     data.append(load_data(info , move))
176
177     #close the gym retro enviornment
178     env.render(close=True)
179     env.close()
180
181     #write data to file

```



```

179     write_to_csv(data, "kdl.csv")
180
181     print("FINISHED!")
182
183 main()

```

Then, we have the **summary_stats.py** file which displays summary statistics to the user and the results are shown in the summary statistics section:

```

1 #Name:          Lucas Hasting
2 #Class:         DA 460
3 #Date:          12/7/2025
4 #Instructor:    Dr. Imbrogno
5 #Description:    Get summary stats of kdl.csv file
6 #Sources:       ChatGPT was used for syntax
7
8 #import libraries
9 import pandas as pd
10
11 #open the data file
12 df = pd.read_csv('kdl.csv')
13
14 #get all columns for screen data
15 columns = []
16
17 for i in range(40):
18     for j in range(4):
19         columns.append(f"screen{i+1}_{j}")
20
21 #run summary stats on screen only data - numeric
22 print(df[columns].stack().describe(), end='\n')
23 print("Name: Screen Data", end='\n\n')
24
25 #get all columns for tile data
26 columns = []
27
28 for i in range(41):
29     for j in range(4):
30         columns.append(f"tile{i+1}_{j}")
31
32 #run summary stats on tile only data - numeric
33 print(df[columns].stack().describe(), end='\n')
34 print("Name: Tile Data", end='\n\n')
35
36 #run summary stats on kirby's scroll x-value data - numeric
37 print(df["kirby_x_scrol"].describe(), end='\n\n')

```

```

38
39 #run summary stats on kirby's scroll y-value data - numeric
40 print(df["kirby_y_scrol"].describe(), end='\n\n')
41
42 #run summary stats on kirby's x-value data - numeric
43 print(df["kirby_x"].describe(), end='\n\n')
44
45 #run summary stats on kirby's y-value data - numeric
46 print(df["kirby_y"].describe(), end='\n\n')
47
48 #run summary stats on boss health data - numeric
49 print(df["boss_health"].describe(), end='\n\n')
50
51 #run summary stats on game state data - nominal
52 print(df["game_state"].value_counts(), end='\n\n')
53
54 #run summary stats on move data - nominal, output/prediction variable
55 print(df["move"].value_counts(), end='\n\n')

```

Next, below we have the **params.py** file which finds the parameters to use for our models:

```

1 #Name:          Lucas Hasting
2 #Class:         DA 460
3 #Date:          12/7/2025
4 #Instructor:    Dr. Imbrogno
5 #Description:    Parameter Search for models.py
6 #Sources:       ChatGPT was used for syntax
7 #               https://scikit-learn.org/stable/api/index.html
8 #               https://mlbenchmarks.org/04-holdout-method.html
9
10 #include data wrangling library
11 import pandas as pd
12
13 #include plotting library
14 import matplotlib.pyplot as plt
15
16 #include model libraries
17 from sklearn.tree import DecisionTreeClassifier
18 from sklearn.neighbors import KNeighborsClassifier
19
20 #include test and accuracy libraries
21 from sklearn.model_selection import train_test_split
22 from sklearn.metrics import accuracy_score
23
24 #min-max norm function
25 def min_max_normalization(x, old_min, old_max, new_min, new_max):

```

```

26         return ((x - old_min) / (old_max - old_min) * (new_max - new_min) +
                new_min)
27
28 #constant for generalization gap threshold
29 GG_THRESHOLD = 0.05
30
31 #get data from csv
32 df = pd.read_csv('kdl.csv')
33
34 #split data into dependent/independent variables
35 y = df["move"]
36 df.drop("move", axis=1, inplace=True)
37
38 #get dummies for game state and normalize
39 df = pd.get_dummies(df, columns=['game_state'], prefix='state', dtype=
    int)
40 for i in df.columns[df.columns.str.startswith('state_')]:
41     lst = df[i].to_list()
42     lst = [min_max_normalization(x, 0, 1, 0, 255) for x in lst]
43     df[i] = pd.DataFrame(lst)
44
45 #min-max norm - kirby_x
46 lst = df["kirby_x"].to_list()
47 lst = [min_max_normalization(x, 0, 65535, 0, 255) for x in lst]
48 df["kirby_x"] = pd.DataFrame(lst)
49
50 #min-max norm - kirby_y
51 lst = df["kirby_y"].to_list()
52 lst = [min_max_normalization(x, 0, 65535, 0, 255) for x in lst]
53 df["kirby_y"] = pd.DataFrame(lst)
54
55 X = df
56
57 #split data into test/training (1/3 - test, 2/3 - training)
58 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3, random_state=42)
59
60 #Find best parameter: max_depth
61 train_accuracy = 1
62 test_accuracy = 0
63 param = 20 #good starting point
64
65 while(abs(train_accuracy - test_accuracy) > GG_THRESHOLD or param == 0)
    :
66     clf = DecisionTreeClassifier(random_state=42, max_depth=param) #
        Initialize the classifier
67     clf.fit(X_train, y_train) # Train the classifier

```

```

68     y_pred = clf.predict(X_test) # Make predictions on the test set
69     accuracy = accuracy_score(y_test, y_pred) # Calculate accuracy
70
71     #check for overfitting using the generalization gap
72     y_train_pred = clf.predict(X_train)
73     y_test_pred = clf.predict(X_test)
74     train_accuracy = accuracy_score(y_train, y_train_pred)
75     test_accuracy = accuracy_score(y_test, y_test_pred)
76     print(f"param: {param}, acc: {abs(train_accuracy - test_accuracy)}
77           = {train_accuracy} - {test_accuracy}")
77     param -= 1
78
79 print(f"Best max_depth: {param+1}\n")
80
81 #Find best parameter: k
82 train_accuracy = 1
83 test_accuracy = 0
84 param = 3 #good starting point
85
86 while(abs(train_accuracy - test_accuracy) > GG_THRESHOLD or param ==
87       100):
87     knn = KNeighborsClassifier(n_neighbors=param, metric='euclidean') #
88         Initialize the classifier
89     knn.fit(X_train, y_train) # Train the classifier
90     y_pred = knn.predict(X_test) # Make predictions on the test set
91     accuracy = accuracy_score(y_test, y_pred) # Calculate accuracy
92
93     #check for overfitting using the generalization gap
94     y_train_pred = knn.predict(X_train)
95     y_test_pred = knn.predict(X_test)
96     train_accuracy = accuracy_score(y_train, y_train_pred)
97     test_accuracy = accuracy_score(y_test, y_test_pred)
98     print(f"param: {param}, acc: {abs(train_accuracy - test_accuracy)}
99           = {train_accuracy} - {test_accuracy}")
100    param += 2
101
102 print(f"Best k: {param - 2}")

```

Finally, below we have the **models.py** file which builds and tests the models:

```

1 #Name:          Lucas Hasting
2 #Class:         DA 460
3 #Date:          12/7/2025
4 #Instructor:    Dr. Imbrogno
5 #Description:    Course Project – Build/Test models for Kirby’s Dream
                  Land

```

```

6 #Sources:          ChatGPT was used for syntax
7 #                  https://scikit-learn.org/stable/api/index.html
8
9 #include data wrangling library
10 import pandas as pd
11
12 #include plotting library
13 import matplotlib.pyplot as plt
14
15 #include model libraries
16 from sklearn.tree import DecisionTreeClassifier, export_text
17 from sklearn.neighbors import KNeighborsClassifier
18
19 #include test and accuracy libraries
20 from sklearn.model_selection import train_test_split
21 from sklearn.metrics import confusion_matrix, accuracy_score,
    ConfusionMatrixDisplay
22
23 #min-max norm function
24 def min_max_normalization(x, old_min, old_max, new_min, new_max):
25     return ((x - old_min) / (old_max - old_min) * (new_max - new_min) +
    new_min)
26
27 #get data from csv
28 df = pd.read_csv('kdl.csv')
29
30 #split data into dependent/independent variables
31 y = df["move"]
32 df.drop("move", axis=1, inplace=True)
33 df = pd.get_dummies(df, columns=['game_state'], prefix='state', dtype=
    int)
34 for i in df.columns[df.columns.str.startswith('state_')]:
35     lst = df[i].to_list()
36     lst = [min_max_normalization(x, 0, 1, 0, 255) for x in lst]
37     df[i] = pd.DataFrame(lst)
38
39 #min-max norm - kirby_x
40 lst = df["kirby_x"].to_list()
41 lst = [min_max_normalization(x, 0, 65535, 0, 255) for x in lst]
42 df["kirby_x"] = pd.DataFrame(lst)
43
44 #min-max norm - kirby_y
45 lst = df["kirby_y"].to_list()
46 lst = [min_max_normalization(x, 0, 65535, 0, 255) for x in lst]
47 df["kirby_y"] = pd.DataFrame(lst)
48
49 X = df

```

```

50
51 #split data into test/training (1/3 - test, 2/3 - training)
52 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3, random_state=42)
53
54 #get count of train/test:
55 print(f"\n\nsize of train: {len(X_train)}, size of test: {len(X_test)}\
    \n\n")
56
57 #Found max_depth in separate program
58 clf = DecisionTreeClassifier(random_state=42, max_depth=17) #
    Initialize the classifier
59 clf.fit(X_train, y_train) # Train the classifier
60 y_pred = clf.predict(X_test) # Make predictions on the test set
61 accuracy = accuracy_score(y_test, y_pred) # Calculate accuracy
62
63 #display confusion matrix
64 fig, ax = plt.subplots(figsize=(200, 200))
65 disp = ConfusionMatrixDisplay(confusion_matrix=confusion_matrix(y_test,
    y_pred), display_labels=sorted(y.unique()))
66 disp.plot(cmap=plt.cm.Blues, ax=ax)
67 disp.ax_.set_xticks([])
68 plt.title('Figure 3: Decision Tree - Confusion Matrix')
69 plt.show()
70
71 #display model accuracy
72 print(f"DT Accuracy: {accuracy:.2f}")
73 print()
74
75 #found K in separate program
76 knn = KNeighborsClassifier(n_neighbors=7, metric='euclidean') #
    Initialize the classifier
77 knn.fit(X_train, y_train) # Train the classifier
78 y_pred = knn.predict(X_test) # Make predictions on the test set
79 accuracy = accuracy_score(y_test, y_pred) # Calculate accuracy
80
81 #display confusion matrix
82 fig, ax = plt.subplots(figsize=(200, 200))
83 disp = ConfusionMatrixDisplay(confusion_matrix=confusion_matrix(y_test,
    y_pred), display_labels=sorted(y.unique()))
84 disp.plot(cmap=plt.cm.Red, ax=ax)
85 disp.ax_.set_xticks([])
86 plt.title('Figure 4: k-NN - Confusion Matrix')
87 plt.show()
88
89 #display model accuracy
90 print(f"KNN Accuracy: {accuracy:.2f}")

```

```
91 print()
92
93 #display Decision Tree
94 print("DECISION TREE:")
95 print(export_text(clf, feature_names = df.columns.tolist()))
```

6 Sources

[1]: https://atacrystal.tcrf.net/wiki/Kirby%27s_Dream_Land/RAM_map.

[2]: <https://github.com/LucasHasting/Prediction-Kirbys-Dream-Land>

[3]: https://retro.readthedocs.io/en/latest/#:~:text=Gym%20Retro%20uses%20emulators%20that%20support%20the,GameGear%20*%20Genesis/Mega%20Drive%20*%20Master%20System

[4]: <https://mlbenchmarks.org/04-holdout-method.html>

[5]: <https://lucashasting.github.io/Prediction-Kirbys-Dream-Land/>